

Call To Adventure

Des données brutes à un catalogue de monstres exploitable

COLLECTER

Des fichiers sources identifiés et conservés.

FIABILISER

Des règles explicites de validation et de fusion.

PARTAGER

Un stockage SQLite et une API REST documentée.

Collecte · SQL · Agrégation · Stockage · API

Une donnée fiable pour un jeu cohérent

Le besoin métier guide les choix techniques.

JOUEUR

Affronter des monstres aux statistiques exploitables : points de vie, armure, difficulté.

DÉVELOPPEMENT

Disposer d'un catalogue normalisé, réutilisable par le jeu et consultable par API.

EXPLOITATION

Reconstruire les données et expliquer leur origine après chaque import.

L'IA raconte ; les données structurées alimentent les règles du jeu.

Un périmètre local et reproductible

Python · Pydantic · SQLite · Django · OpenAPI

CONTRAINTES

Quelques centaines de monstres.
Exécution locale. Démonstration
du pipeline sans appel au LLM.

ORGANISATION

Projet solo avec assistance
outillée. Étapes : collecte, qualité,
stockage, exposition, preuves.

À PERSONNALISER

Nom et rôle exacts ; dates des
étapes ; temps réellement passé ;
budget engagé.

Le budget et le calendrier réels ne sont pas chiffrés dans les documents disponibles.

Une chaîne de données traçable

Chaque étape produit une donnée ou une preuve inspectable.

01 / SOURCES

Snapshot collecté + fichier corrigé



Chargement et métadonnées

02 / QUALITÉ

Normalisation + validation



Fusion, rejets, manifeste

03 / DIFFUSION

SQLite + provenance



Requêtes SQL et API Django

Brut conservé → jeu nettoyé → catalogue partagé

Deux entrées, une origine à expliciter

Les snapshots sont reproductibles ; la collecte web reste un sujet distinct.

SNAPSHOT EXTERNE

monster_scrapping/monsters.json
510 enregistrements
Statistiques issues de la collecte.

SOURCE CORRIGÉE

data/documents/monsters.json
510 enregistrements
Adaptations aux besoins du jeu.

CONTRAINTES WEB

Scraper documenté :
temporisation, timeout, retries et
robots. Preuve hors ligne par
fixture.

Les deux fichiers sont liés : leur acceptation comme sources distinctes reste à confirmer.

Transformer sans perdre le sens

Exemple pédagogique repris des cas de test.

AVANT

Nom : Élite Goblin
HP : "12"
Difficulté : "1/4"
Description : " test "

APRÈS

Clé : elite-goblin
HP : 12 (entier)
Difficulté : 0,25 + texte "1/4"
Description : "test"

CONTRÔLES

Nom, armure et HP obligatoires.
HP strictement positifs.
Erreur → rejet avec motif
VALIDATION_ERROR.

Un enregistrement invalide est isolé et conservé pour diagnostic.

Résoudre les conflits de façon déterministe

Même clé stable → un seul monstre canonique.

SOURCE COLLECTÉE

Exemple de test : Goblin
HP = 10
Provenance conservée.

SOURCE CORRIGÉE

Exemple de test : Goblin
HP = 20
Priorité aux données corrigées.

RÉSULTAT

Goblin : HP = 20
Deux liens de provenance.
Source gagnante marquée
selected.

La priorité est une règle métier explicite, pas une preuve de vérité de la source.

1 020 entrées deviennent 510 monstres

Exécution locale du 14 septembre 2026 · snapshots du dépôt

1 020

Enregistrements acceptés
1 020 collectés
0 rejet sur ces snapshots

510

Monstres après fusion
5 conflits détectés
Priorité à la source corrigée

5 / 5

Tests du pipeline réussis
Normalisation, fractions, fusion,
provenance et sorties JSONL

Le succès du run ne démontre pas, à lui seul, la couverture complète de C1 à C5.

Extraire une donnée utile et explicable

Quatre requêtes versionnées couvrent les usages du catalogue.

Q1 / Q2

Q1 : sélectionner un catalogue valide et trié.

Q2 : joindre chaque monstre à ses sources via un ID paramétré.

Q3 / Q4

Q3 : agréger le bilan d'import.

Q4 : regrouper par difficulté et calculer les moyennes HP / armure.

INDEX

Nom insensible à la casse.

Difficulté.

Liens de provenance.

Plans via EXPLAIN QUERY PLAN.

Les index répondent aux accès ; aucun gain de performance non mesuré n'est annoncé.

Séparer le catalogue et les données de compte

Vue simplifiée du modèle ; MCD et MLD complets dans la documentation.

IMPORT

INGESTION_RUN
1 → N MONSTER
1 → N REJECTED_RECORD
1 → N MONSTER_SOURCE

PROVENANCE

MONSTER
1 → N MONSTER_SOURCE

Clés étrangères, unicité de
stable_key, contraintes HP /
armure.

DONNÉES PERSONNELLES

USER
1 → N SAVE_GAME
1 → N CHARACTER_TEMPLATE

Base Django séparée du
catalogue.

Chaque donnée a un rôle, des contraintes et une relation définis.

SQLite répond au volume de la démonstration

Un choix proportionné, avec des limites opérationnelles identifiées.

POURQUOI SQLITE

Fichier local, peu de données, import par un seul processus et lectures fréquentes.

RECONSTRUCTION

Sources versionnées + pipeline. Schéma créé par le code. Import transactionnel avec provenance.

LIMITES

Le catalogue est remplacé à chaque import. Historique complet non garanti. PostgreSQL à envisager pour plusieurs écrivains.

Les données de compte nécessitent une sauvegarde distincte et une restauration testée.

Documenter les traitements et les droits

État du registre et des procédures du projet.

PÉRIMÈTRE

Comptes, e-mails, mots de passe hachés, sauvegardes et personnages rattachés à un compte.

PROCÉDURES

Export utilisateur.
Rectification via administration.
Suppression du compte et cascades.
Nettoyage avec prévisualisation.

À VALIDER

Bases légales et durées proposées.
Sauvegardes terminées : 365 jours.
Logs : 30 jours proposés.
Validation par le responsable.

Un registre et des commandes constituent des éléments de démarche ; pas une certification de conformité.

Partager le catalogue avec un contrat clair

API en lecture seule · version v1 · documentation OpenAPI

LECTURE PUBLIQUE

GET /api/v1/monsters/
GET /api/v1/monsters/{id}/

Filtres, tri et pagination
100 résultats maximum par page.

ACCÈS RESTREINT

Résumé d'import :
/api/v1/ingestion-runs/
{run_id}/summary/

Session Django + compte staff.

CONTRAT

/api/v1/docs/
/api/v1/openapi.yaml

Paramètres validés ; SQL
paramétré ; erreurs structurées.

Public pour le catalogue ; réservé au personnel pour les informations d'import.

Suivre un résultat de bout en bout

Séquence proposée : 1 min 45, avec preuves hors ligne en secours.

1 / MANIFESTE

Afficher les compteurs :
1 020 → 510
5 conflits, 0 rejet.
Ouvrir un objet nettoyé.

2 / PROVENANCE

Montrer le test de fusion.
Puis Q2 : un monstre, ses sources
et la source sélectionnée.

3 / API

Ouvrir la collection filtrée.
Afficher le détail d'un monstre.
Relier les champs au contrat
OpenAPI.

Préparer la base et le serveur avant l'oral ; conserver le manifeste comme solution de secours.

Un socle concret, des écarts à traiter

La couverture technique doit rester distincte de la validation par le jury.

DÉMONSTRABLE

Pipeline sur fichiers, normalisation, fusion et persistance testées. SQL, modèles et contrat API présents.

ÉCARTS E1

Aucune source big data.
Pas de collecte depuis une API externe.
Deux snapshots liés.
Collecte web directe non démontrée.

AVANT L'ORAL

Compléter budget et planning.
Répéter API et restauration.
Valider les choix de sources.
Rapprocher chaque preuve du référentiel.

Le résultat : un catalogue exploitable dont les transformations sont explicables.

Revenir à l'origine d'un monstre

Extrait exact de Q2, catalogue SQL du dépôt.

SÉLECTION

```
SELECT m.name, s.source,  
s.source_record_id, s.source_url,  
s.collected_at, s.selected,  
s.ingestion_run_id
```

JOINTURE

```
FROM monsters AS m  
JOIN monster_sources AS s  
ON s.monster_id = m.id
```

PARAMÈTRE ET TRI

```
WHERE m.id = :monster_id  
ORDER BY s.selected DESC,  
s.source;
```

ID lié comme paramètre SQL.

Le résultat explique à la fois la provenance et la priorité de fusion.

Où retrouver les éléments du dossier

Les chemins sont relatifs à la racine du dépôt.

C1 / C3

src/data_pipeline/
tests/test_data_pipeline.py
docs/block1/data-pipeline.md
preuves/<run>/manifest.json

C2 / C4

db/sqlite/queries/
docs/block1/data-models.md
docs/block1/database-and-sql.md
docs/block1/gdpr-register.md

C5 / CADRAGE

docs/block1/openapi.yaml
docs/block1/api-security.md
certif/*Referentiel*.pdf
certif/*Règlement*.pdf

Preuves générées dans certif/presentation-e1/preuves/.

Défendre les choix et leurs limites

Quatre questions probables du jury.

POURQUOI CES SOURCES ?

Elles répondent au besoin du jeu.
Leur lien est déclaré ; les
catégories de sources manquantes
restent un écart.

POURQUOI SQLITE ?

Volume modeste et démo locale.
Plusieurs écrivains ou instances
conduiraient à revoir le SGBD.

ET LA QUALITÉ ?

Règles déterministes, rejets
conservés, provenance et tests.
Aucun rejet sur un run ne signifie
pas absence de défaut possible.

Pourquoi l'API est publique ? Elle expose un catalogue de référence ; les résumés d'import sont restreints.